

Postman AIML Task WriteUp

Name: Anirudh Jutu

Task Description: Building a small artificial neural network and implementing backpropagation manually, using NumPy and Scikit-Learn.

Introduction

This project involves the creation of a small feedforward neural network implemented from scratch, along with manually derived forward and backward propagation using linear layers and chain rule. Additionally, gradients, both analytical and numerical were calculated and checked using numerical gradient checking, and the results matched with reasonable tolerance.

Moreover, the network was then trained on a small task of reduced MNIST dataset, using the scikit-learn library. Finally, the stretch goal was achieved with the help of Adam, where another activation loss combination function was used to derive the same gradients as before.

1.1 Feedforward Neural Network

Output:

```

Input values of x:
[[0.1 0.2 0.3]
 [0.2 0.3 0.4]
 [0.3 0.4 0.5]
 [0.4 0.5 0.6]
 [0.6 0.7 0.8]]
Hidden Layer output after rectifying linear unit:
[[0.00241627 0.06663155 0.02959137 0.02399481 0.04020345]
 [0.01172445 0.09467674 0.04547555 0.0465883 0.06742364]
 [0.02103263 0.12272193 0.06135974 0.06918179 0.09464384]
 [0.03034081 0.15076712 0.07724392 0.09177529 0.12186404]
 [0.04895717 0.2068575 0.10901229 0.13696227 0.17630443]]
Final output of the network:
[[-0.00522918 0.00609178]
 [-0.00739405 0.01243666]
 [-0.00955893 0.01878154]
 [-0.01172381 0.02512643]
 [-0.01605356 0.0378162 ]]

```

Program Completion Date: 12th September

Program Description:

This program demonstrates a simple feedforward neural network, where basic matrix operations are used.

An input matrix is linearly layered with a weight and bias matrix, followed by linear rectification, before feedforwarding it to the next layer from the current hidden layer, resulting in the given output.

Errors Faced:

Tried performing matrix multiplication using the usual asterisk operator

Fix:

Learned that matrix multiplication using numpy arrays requires the '@' symbol.

1.2 Implementation of Forward Pass

Output:

```

undted/ C:\BS/debugpy/adapter/ ../ ../debugpy/ launcher
Input x:
[[0.1 0.2 0.3]
 [0.2 0.3 0.4]
 [0.3 0.4 0.5]
 [0.4 0.5 0.6]]
Hidden layer output after rectifying linear unit:
[[0.05189512 0.          0.03311046 0.06300999]
 [0.08717903 0.          0.05383916 0.09844809]
 [0.12246295 0.          0.07456785 0.13388618]
 [0.15774686 0.          0.09529655 0.16932428]]
Final output of the network:
[[ 0.00910445  0.0032503  -0.01082601]
 [ 0.01475493  0.00497258 -0.017182   ]
 [ 0.02040542  0.00669487 -0.02353798]
 [ 0.02605591  0.00841715 -0.02989397]]

```

Program Completion Date: 12th September

Program Description:

This program is very similar to the previous one, where the linear layer is implemented as a discrete function and the RLU function acting as the activation function.

Errors Faced:

1. Performed $w @ x$ instead of $x @ w$ for the linlay function
2. Fed the original input of w and b again for the second layer

Fix:

1. Changed it to $x @ w$ appropriately
2. Replaced it with $w1$ and $b1$

1.3 Backward pass and chain rule implementation

Output:

```

undled/libs/debugpy/adapter/../../debugpy/launcher 61108 -- /Use
Forward pass output:
[[-0.00522918  0.00609178]
 [-0.00739405  0.01243666]
 [-0.00955893  0.01878154]]
Mean Squared Error Loss: 0.49145664306038067
dL/dw (shape (3, 5) ):
[[-0.0256329 -0.00440952  0.04479244 -0.01644051 -0.03457432]
 [-0.03654453 -0.00575867  0.06440501 -0.02432297 -0.04689498]
 [-0.04745617 -0.00710783  0.08401757 -0.03220543 -0.05921564]]
dL/db (shape (5, ) ):
[-0.10911633 -0.01349152  0.19612564 -0.07882459 -0.12320656]
dL/dw1 (shape (5, 2) ):
[[-0.00090555 -0.01073384]
 [-0.02295104 -0.07117013]
 [-0.01022296 -0.03497901]
 [-0.00837535 -0.03791506]
 [-0.01393897 -0.05306883]]
dL/db1 (shape (2, ) ):
[-0.34072739 -0.65423001]

```

Program Completion Date: 13th September

Program Description: This program implements manual backward propagation for the same feedforward propagation used in the previous program. However, the utilisation of autograd or .backward()

Errors Faced:

1. Wrong formula while calculating the loss gradient
2. Did not find the transpose matrix while calculating the weight and bias gradients
3. During bias gradient calculation, put axis = 1 instead of 0 (believed axis = 1 is for column addition, just as it is for inserting a column in an np array)

Fix:

1. Learned the formula using claude.ai, and implemented the right one in the program.
2. Learned the purpose of taking transpose (to satisfy the shape) and implementing it accordingly.
3. Learned it is actually axis = 0 for the addition of value in a particular column.

1.4 Gradient checking

Output:

```
und ted/ libs/ debugpy/ adapter/ ../ ../ debugpy/ taun  
Analytic gradient:  -0.00664977368155471  
Numerical gradient:  -0.006649773681699499  
Difference:  1.447895622841422e-13
```

Completion Date: 14th September

Program Description: Checks gradients against numerical gradient checking, and checking if results match.

Errors Faced: Reusing of x and y at the end of the program, assigned y = dw_num[0][0] when y wasn't even a numpy array

Fix: Changed it to y = dw_num accordingly

1.5 Training network using scikit-learn datasets

Output:

```
Enter the number of rounds: 500
round: 10
loss: 2.1207141252699806
round: 20
loss: 1.6199914602893417
round: 30
loss: 0.9764491741978465
round: 40
loss: 0.6221454451259106
round: 50
loss: 0.4546612409106255
round: 60
loss: 0.3618310848608931
round: 70
loss: 0.3028953890649716
round: 80
loss: 0.26212963731894207
round: 90
loss: 0.23229912707032757
round: 100
loss: 0.20952278954127615
round: 110
loss: 0.19153888085773052
round: 120
loss: 0.1769295471086204
round: 130
loss: 0.1648499404425596
round: 140
loss: 0.15469077766215497
round: 150
loss: 0.14602521861519582
round: 160
loss: 0.13851953165013717
round: 170
loss: 0.13195403868466268
round: 180
loss: 0.12615252905658758
round: 190
loss: 0.12097569987184618
```

Completion Date: 14th September

Program Description: Train gradients using scikit-learn datasets and check if the loss decreases over time in consecutive rounds, as seen in the output.

Errors faced:

1. Didn't include keepdims = True in the sigmoid calculator

Fix:

1. Understood the importance of keepdims = True (to retain the matrix dimensions, when division is being performed for sigmoid) and implemented it accordingly.

1.6 Gradient mistake made and rectification

Errors Faced:

1. Did not find the transpose matrix while calculating the weight and bias gradients
2. During bias gradient calculation, put axis = 1 instead of 0 (believed axis = 1 is for column addition, just as it is for inserting a column in an np array)

Fix:

1. Learned the purpose of taking transpose (to satisfy the shape) and implementing it accordingly.
2. Learned it is actually axis = 0 for the addition of value in a particular column.

Stretch Goal

Output:

```
undted/.../debugpy/adapter/.../debugpy/... 01540
Enter the digit to be identified using neural networks: 3
round: 20
loss: 0.3129959100047451
round: 40
loss: 0.2082400793868908
round: 60
loss: 0.14685288931023827
round: 80
loss: 0.12546598211175167
round: 100
loss: 0.11226072871739137
round: 120
loss: 0.10199426498583214
round: 140
loss: 0.0933131102481715
round: 160
loss: 0.08545179861325111
round: 180
loss: 0.07836666917316586
round: 200
loss: 0.07201718575436462
round: 220
loss: 0.06633609462197568
round: 240
loss: 0.061262572513759966
round: 260
loss: 0.05669426179881926
round: 280
loss: 0.05253619916768532
round: 300
loss: 0.04874144849759621
```

Completion Date: 14th September

Program Description: Additional activation loss combination and gradient derivation using the implementation of Adam.

Errors faced

1. Using for 'i' in pr again instead of for 'j' in pr, leading to clashing of

- variables for inner loop
- 2. Wrong formula for loss towards the end

Fix:

- 1. Introduced 'j' variable for inner loop
- 2. Replaced it with the correct formula